

```
In [1]: import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt
from time import time
```

```
In [20]: df1= pd.read_csv('train_data_winsorized.csv')
df2= pd.read_csv('test_data_unprocessed.csv')

train_dataset = np.array(df1)
val_dataset = np.array(df2)

#Scale the data between 0 and 1
scaler = MinMaxScaler((0,1))
train_dataset_scaled = scaler.fit_transform(train_dataset)
test_dataset_scaled = scaler.transform(test_dataset)

#Inputs
x_train = train_dataset_scaled[:,0:5]
x_val = test_dataset_scaled[:,0:5]

#Outputs
y1_train = train_dataset_scaled[:,5] #y1 - Adsorption Rate
y1_val = test_dataset_scaled[:,5]
y2_train = train_dataset_scaled[:,6] #y2 - Desorption Rate
y2_val = test_dataset_scaled[:,6]

feature_names = ['T[K]', 'Humidity (%)', 'AdsTime[s]', 'DesTime[s]', '
output_names = ['q_ads[mol/kg]', 'q_des[mol/kg]']
```

```
In [22]: from sklearn.linear_model import LinearRegression
from sklearn.metrics.pairwise import rbf_kernel
from sklearn.model_selection import train_test_split
import numpy as np

# Sample 20% of the training data (Hold-out)
# Using the rest 80% to optimize the hyperparameter
x_train_sample, x_test_sample, y1_train_sample, y1_test_sample, y2_train_sample, y2_test_sample = train_test_split(
    x_train, x_val, y1_train, y1_val, y2_train, y2_val, test_size=0.2, random_state=42)

print("20% of training set was sampled for the fitting")

gamma_values = [0.1, 0.05, 0.01, 0.005, 0.001]

best_gamma_ads = None
best_score_ads = -100
y1_pred_best = None
best_model_ads = None

start_time = time()
```

```

for gamma in gamma_values:
    X_train_rbf = rbf_kernel(x_train_sample, gamma=gamma)
    X_test_rbf = rbf_kernel(x_test_sample, x_train_sample, gamma=gamma)

    model_rbf_ads = LinearRegression()
    model_rbf_ads.fit(X_train_rbf, y1_train_sample)
    y1_pred = model_rbf_ads.predict(X_test_rbf)
    ads_score = r2_score(y1_test_sample, y1_pred)
    if ads_score > best_score_ads:
        best_score_ads = ads_score
        best_gamma_ads = gamma
        y1_pred_best = y1_pred
        best_model_ads = model_rbf_ads

print("<Adsorption Model>")
print(f"LinearRegression using RBF kernel fit done in {(time() - start_time):.10f}s")
print(f"Optimal gamma: {best_gamma_ads} with score: {best_score_ads}")

best_gamma_des = None
best_score_des = -100
y2_pred_best = None
best_model_des = None

start_time = time()
for gamma in gamma_values:
    X_train_rbf = rbf_kernel(x_train_sample, gamma=gamma)
    X_test_rbf = rbf_kernel(x_test_sample, x_train_sample, gamma=gamma)

    model_rbf_des = LinearRegression()
    model_rbf_des.fit(X_train_rbf, y2_train_sample)
    y2_pred = model_rbf_des.predict(X_test_rbf)
    des_score = r2_score(y2_test_sample, y2_pred)

    if des_score > best_score_des:
        best_score_des = des_score
        best_gamma_des = gamma
        y2_pred_best = y2_pred
        best_model_des = model_rbf_des

print("<Desorption Model>")
print(f"LinearRegression using RBF kernel fit done in {(time() - start_time):.10f}s")
print(f"Optimal gamma: {best_gamma_des} with score: {best_score_des}")

```

20% of training set was sampled for the fitting

<Adsorption Model>

LinearRegression using RBF kernel fit done in 34.23447871208191s

Optimal gamma: 0.05 with score: 0.9596093996822044

<Desorption Model>

LinearRegression using RBF kernel fit done in 34.14204287528992s

Optimal gamma: 0.05 with score: 0.9745700506709071

In [23]: *#Evaluating r2 score with validation set (unwinsorized test set)*
X_val_rbf = rbf_kernel(x_val, x_train_sample, gamma=best_gamma_ads)

```

y1_val_pred = best_model_ads.predict(X_val_rbf)
r2_ads = best_model_ads.score(X_val_rbf, y1_val) #get the "score", whi
print('Validation set r^2 q_ads= {}'.format(r2_ads))

X_val_rbf = rbf_kernel(x_val, x_train_sample, gamma=best_gamma_des)
y2_val_pred = best_model_des.predict(X_val_rbf)
r2_des = best_model_des.score(X_val_rbf, y2_val) #get the "score", whi
print('Validation set r^2 q_des= {}'.format(r2_des))

#Parity plot
z=[0,1]
w=[0,1]
fig, axes= plt.subplots(2, figsize=(6, 10))
axes[0].scatter(y1_test, y1_val_pred, alpha=0.3)
axes[0].plot(z, w, 'k--') #y=x line
axes[0].set_xlabel('Actual', fontsize=12)
axes[0].set_ylabel('Predicted', fontsize=12)
axes[0].set_title('q_ads', fontsize=10)

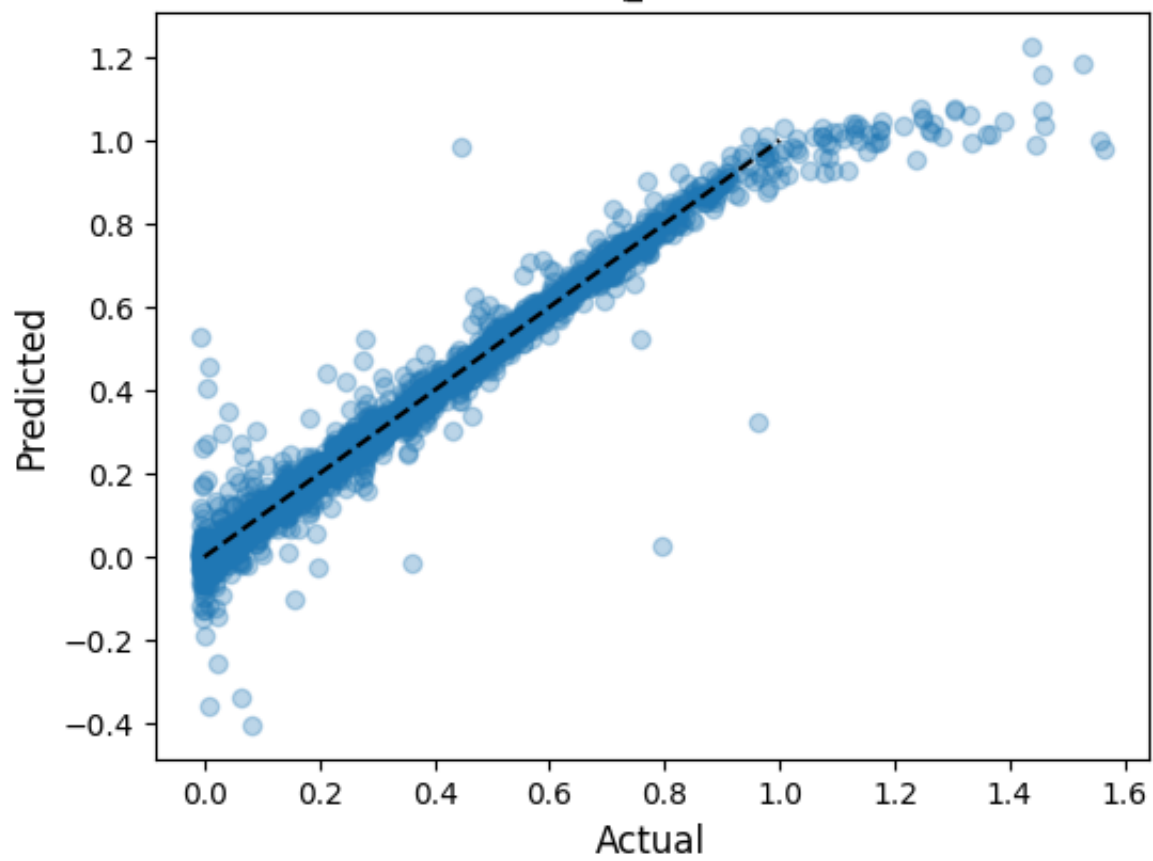
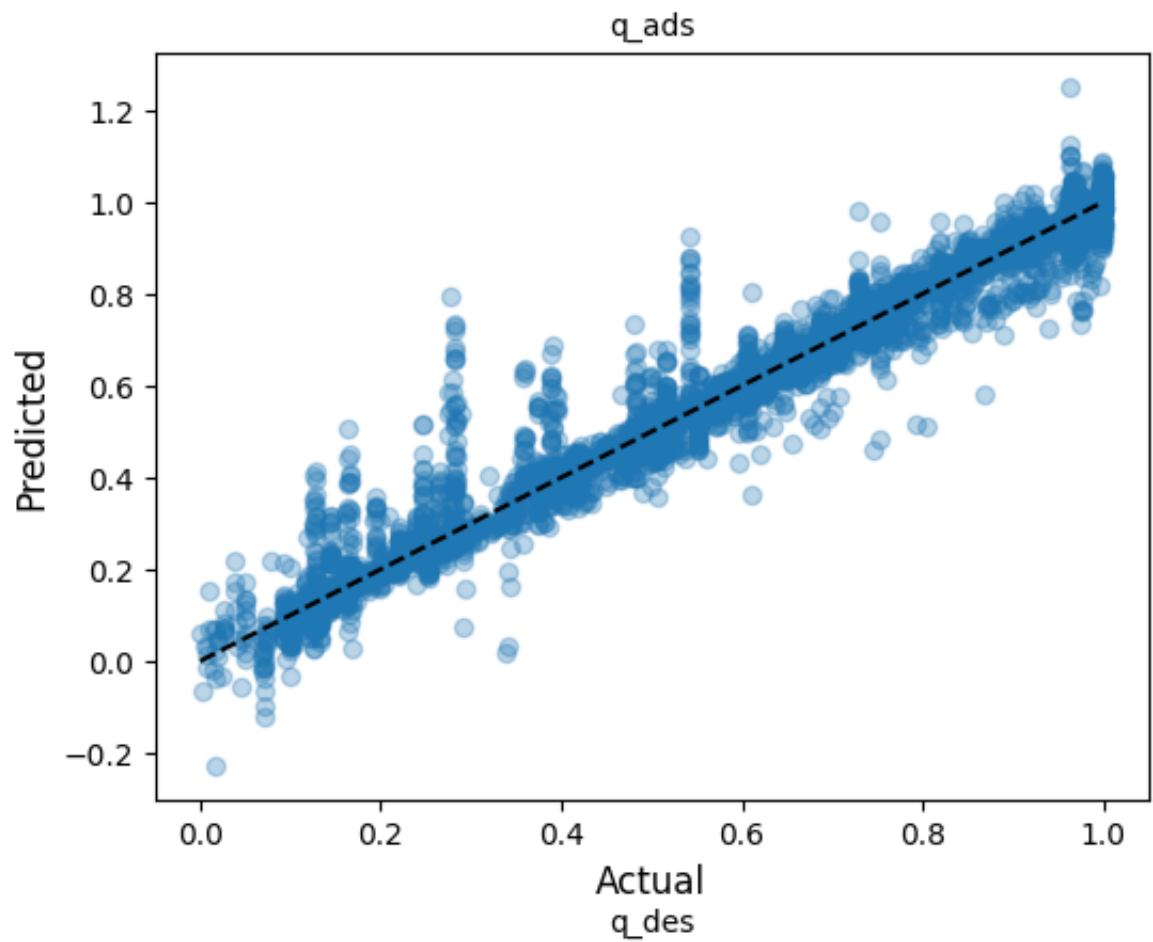
axes[1].scatter(y2_test, y2_val_pred, alpha=0.3)
axes[1].plot(z, w, 'k--') #y=x line
axes[1].set_xlabel('Actual', fontsize=12)
axes[1].set_ylabel('Predicted', fontsize=12)
axes[1].set_title('q_des', fontsize=10)

```

Validation set r^2 q_ads= 0.9538369005678309

Validation set r^2 q_des= 0.9642757277072288

Out[23]: Text(0.5, 1.0, 'q_des')



In []: